IAEA-NDS-0244
Rev. 2026/10



INTERNATIONAL ATOMIC ENERGY AGENCY

NUCLEAR DATA SERVICES

DOCUMENTATION SERIES OF THE IAEA NUCLEAR DATA SECTION

ForEXy: Utility Codes for EXFOR Library

Naohiko Otuka
IAEA Nuclear Data Section, Vienna, Austria

October 2026

IAEA Nuclear Data Section, Vienna International Centre, A-1400 Vienna, Austria

Note:

The IAEA-NDS-reports should not be considered as formal publications. When a nuclear data library is sent out by the IAEA Nuclear Data Section, it will be accompanied by an IAEA-NDS-report which should give the data user all necessary documentation on contents, format and origin of the data library.

IAEA-NDS-reports are updated whenever there is additional information of relevance to the users of the data library.

For citations care should be taken that credit is given to the author of the data library and/or to the data centre which issued the data library. The editor of the IAEA-NDS-report is usually not the author of the data library.

Neither the originator of the data libraries nor the IAEA assume any liability for their correctness or for any damages resulting from their use.

96/11

Citation guideline:

When quoting this code package in a publication this should be done in the following way:

N. Otuka, V. Devi, O. Iwamoto "EXFOR utility codes (ForEXy) and their application to neutron fission cross section evaluation", Applied Radiation and Isotopes **225** (2025) 111903. (https://doi.org/10.1016/j.apradiso.2025.111903).

IAEA-NDS-0244
Rev. 2026/09

ForEXy: Utility Codes for EXFOR Library

Naohiko Otuka
IAEA Nuclear Data Section, Vienna, Austria

Abstract

Descriptions are given for a package of utility codes for the files relevant to EXFOR library. This program package is written in Python and may be downloaded from the NRDC software website (http://nds.iaea.org/nrdc/nrdc_sft/).

September 2026

Contents

Introduction 7
Acknowledgements 8
History (major revisions only) 8
C6TOCX 11
CODUSE 12
DIC227 13
DICA2J 14
DICDIS 15
DICJ2A 16
DICJ2T 17
DIRINI 18
DIRUPD 20
EXTMUL 21
J4TOC6 22
J4TOX4 24
J4VIEW 25
LOTRAN 26
MAKC6L 28
MAKCOV 30
MAKCXL 32
MAKJ4L 34
MAKLIB 35
PLOTCX 36
POIPOI 37
REFBIB 39
REFDOI 40
SEQADD 41
SPELLS 42
X4TOC6 43
X4TOCX 45
X4TOJ4 47
X4VIEW 49
Appendix 1: Output of formatted free text to HTML by J4VIEW / X4VIEW 50
Appendix 2: Quantity and reaction specifications in C4 and C6 formats 52
Appendix 3: Standard file naming convention 56

Appendix 4: HED file used in MAKCOV 58
Appendix 5: Use a ForEXy code as a module 63

Introduction

The Utility Codes for EXFOR Library (ForEXy) are written to process EXFOR Entry files and EXFOR/CINDA Dictionary files. Currently, the following 29 codes (Python scripts) are included in this package:

- C6TOCX Convert a C6 file to a CX4 file
- CODUSE Analyse use of codes in J4 library.
- DIC227 Produce Archive Dictionary 227 from a NUBASE file.
- DICA2J Convert Archive dictionaries to a JSON Dictionary.
- DICDIS Prepare Archive and Backup dictionaries for distribution.
- DICJ2A Convert a JSON Dictionary to Archive dictionaries.
- DICJ2T Convert a JSON Dictionary to a Transmission dictionary.
- DIRINI Split an EXFOR library tape into EXFOR entry files.
- DIRUPD Update the EXFOR entry files with an EXFOR transmission tape.
- EXTMUL Extract a dataset from a multiple reaction formalism subentry.
- J4TOC6 Convert a J4 file to a C6 or C4 file.
- J4TOX4 Convert a J4 file to an EXFOR file.
- J4VIEW Convert a J4 file to an HTML file.
- LOTRAN Lorentz transformation for a differential cross section data point
- MAKC6L Produce a C6 or C4 library
- MAKCOV Produce a data table and covariance matrix from a J4 file.
- MAKCXL Produce a CX4 library
- MAKJ4L Produce and update a J4 library.
- MAKLIB Merge EXFOR entry files into a single library tape.
- PLOTCX Plot CX4 file by gnuplot
- POIPOI Remove pointers from a J4 file.
- REFBIB Extract bibliography of reference by using DOI.
- REFDOI Obtain DOI for articles registered in CrossRef.

- SEQADD Add record identification to an EXFOR file.
- SPELLS Check English spell in free text in EXFOR format.
- X4TOC6 Convert an EXFOR file to a C6 file.
- X4TOCX Convert an EXFOR file to a CX4 file.
- X4TOJ4 Convert an EXROF file to a J4 file.
- X4VIEW Convert an EXFOR file to an HTML file.

("J4" means EXFOR in JSON. "CX4" means Compact EXFOR file.)

This package is distributed in a zipped form from the NRDC software website (https://nds.iaea.org/nrdc/nrdc_sft/) and the PyPI repository (https://pypi.org/project/forexy/).

This document explains how to use these codes. Users need to install Python3 in their environments prior to run these codes. Any comments on the use of the codes, including difficulties encountered or any possible bug reports and suggestions are welcome.

Optional arguments available in all codes

- `-h` Display help information
- `-v` Display the version
- `-f` never prompt

Acknowledgements

The developer would like to thank David Brown, Oscar Cabellos, Georg Schnabel and Nicolas Soppera for their comments and proposals.

History (major revisions only)

2023-10-23:

- First release of 4 scripts (DIRINI, DIRUPD, MAKLIB, SEQADD)

2023-11-02:

- First release of IAEA-NDS-0244.

2024-05-03:

- First release of 6 scripts (DIC227, DICA2J, DICDIS, DICJ2A, DICJ2T, SPELLS)

2024-06-25:

- Addition of -c option to MAKLIB.
- Update of DICA2J to implement format changes of Dictionaries 25, 209 and 227 concluded in the NRDC 2024 meeting (C12 and C13).

2024-10-07:

- First release of 4 scripts (J4TOX4, MAKTAB, POIPOI, X4TOJ4).

2024-11-21:

- First release of EXTMUL. MAKTAB renamed into MAKCOV.

2025-01-05:

- First release of REFBIB and REFDOI.

2025-03-31:

- The package name was renamed from ForEX to ForEXy.
- DIRUPD updated. Now it does not require a log file (e.g., "x4_dirupd.log") generated in a previous initialization by DIRINI or update by DIRUPD.

2025-12-01:

- First release of CODUSE and MAKJ4L.
- One argument (`keepres0`) was added in the main function of POIPOI.
- The field name "radiation" was changed to "radiation_type" for the EXFOR keyword RAD-DET in J4.

2026-01-26:

- -m was changed to -n for controlling of addition of "19" in SEQADD.
- One argument (`email0`) was added in the main function of X4TOJ4.
- The field name "doi" was added to the EXFOR keyword REFRENCE in J4.
- -e was changed to -m for providing an email address to REFBIB and REFDOI.

2026-05-25:

- First release of J4VIEW and X4VIEW.
- One argument (`file_log`) was added in the main function of X4TOJ4.

2026-09-30:

- First release of C6TOCX, J4TOC6, LOTRAN, MAKC6L, MAKCXL, PLOTCX, X4TOC6 and X4toCX.

- One argument (`file_log`) was removed from the main function of X4TOCX.

2026-10-02:

- One argument (`entry`) was added in the main function of X4TOJ4.

2026-10-14:

- The arguments `file_c6` (C6 file name) and `file_cx4` (CX4 file name) were changed to `dir_c6` (C6 file directory name) and `dir_cx4` (CX4 file directory name), respectively, and also two arguments (`entry`, `exclhi`) were added in the main function of C6TOCX. One argument (`entry`) was added in the main function of X4TOJ4, and one argument (`plot`) was added in the main function of X4TOCX.

C6TOCX

This code reads a C6 file and converts it to a CX4 file.

Input files

- directory of C6 files following standard C6 naming convention (`-i`) [1]
- JSON Dictionary (`-d`)

Output file

- CX4 file (`-o`)
- index file (`-n`)

Arguments

- `-i` *`dir_c6`* input C6 file directory
- `-d` *`file_dict`* input JSON Dictionary (optional, default: "*dict.json*")
- `-e entry` EXFOR Entry number or "all" to process all C6 files in the input directory
- `-o` *`dir_cx4`* output CX4 file directory
- `-n` *`file_idx`* output index file (optional, default: "*x4_c6tocx.txt*")
- `-s` *`sysout`* output reference system identifier (optional, default: "OOO")[2]
- `-x` exclude heavy-ion (A>4) induced reactions

Example

Convert a C6 file of EXFOR Entry *10003.002-n-Ti-tot-sig-excfun-mon-Wang_1974.c6* to a CX4 file following the standard CX4 file naming:

```
python3 x4_c6tocx.py -i 10003.002-n-Ti-tot-sig-excfun-mon-Wang_1974.c6
-a
```

[1] See Appendix 3 for the standard file naming convention.
[2] See LOTRAN for the reference system identifier.

CODUSE

This code reads a J4 file storage (produced by MAKJ4L) and summarizes (1) codes used in an EXFOR entry but not defined in EXFOR/CINDA Dictionary, and (2) codes defined in EXFOR/CINDA Dictionary but not used in EXFOR and CINDA.

Input files

- J4 files (`-i`)
- JSON Dictionary (`-d`)
- CINDA code list (`-c`)

Output file

- log file (`-l`)

Arguments

- `-i` *`dir_storage`* directory of input J4 file storage
- `-d` *`file_dict`* input JSON Dictionary (optional, default: "*dict.json*")
- `-c` *`file_cinda`* input list of codes used in CINDA (optional, default: "*x4_coduse.cin*")
- `-l` *`file_log`* output log file (optional, default: "*x4_coduse.log*")

Example

Read J4 files in the J4 file storage *j4*:

```
python3 x4_coduse.py -i j4
```

DIC227

This code reads a NUBASE file and a supplemental dictionary file (compilation of properties of elementary particles and natural elements in the Archive Dictionary format), and converts them to Archive Dictionary227.

Input files

- NUBASE file[3] (`-i`)
- supplemental file (`-s`)

Output file

- Archive Dictionary 227 file (`-o`)

Arguments

- `-i` *`file_nubase`* input NUBASE file
- `-s` *`file_suppl`* input supplemental dictionary file (optional, default: "*dict_arc_sup.227*")
- `-o` *`file_arc227`* output archive dictionary file (optional, default: "*dict_arc_new.227*")

Example

Convert a NUBASE file *nubase_4.mas20.txt*:

```
python3 x4_dic227.py -i nubase_4.mas20.txt
```

[3] The NUBASE file is distributed from the Atomic Mass Data Center (https://amdc.impcas.ac.cn/, https://www.anl.gov/phy/atomic-mass-data-resources, https://nds.iaea.org/amdc/).

DICA2J

This code reads Archive Dictionaries and converts them to a JSON Dictionary.

Input file

- Archive Dictionaries (`-i`)

Output file

- JSON Dictionary (`-o`)

Arguments

- `-n` *`dict_ver`* dictionary version (transmission ID)
- `-i` *`dir_archive`* directory of input Archive Dictionaries
- `-o` *`dir_json`* directory of output JSON Dictionary

Example

Convert Archive Dictionaries *input/dict_arc.top, input/dict_arc_new.001* etc. to a JSON Dictionary *json/dict.json*.

```
python3 x4_dica2j.py -n 9131 -i input -o json
```

DICDIS

This code reads Archive and JSON Dictionaries and process them for distribution after removal of records with the alteration flag D (deletion) and updating the year + month field (YYYYMM). At the same time, this code also produces the Backup Dictionary.

Input files

- Archive Dictionaries (`-a`)
- JSON Dictionary (`-j`)

Output files

- Archive, Backup and JSON Dictionaries for distribution (`-o`)

Arguments

- `-n` *`dict_ver`* dictionary version (transmission ID)
- `-a` *`dir_archive`* directory of input Archive Dictionaries
- `-j` *`dir_json`* directory of input JSON Dictionary
- `-o` *`dir_dist`* directory of output dictionaries

Example

Read Archive and JSON Dictionaries *input/dict_arc.top, input/dict_arc_new.001* etc. and *json/dict.json*, process them for distribution, and output them under the same names but under the directory *dist*. At the same time, it also produces the Backup Dictionary *dist/dan_back_new.9131*.

```
python3 x4_dicdis.py -n 9131 -a input -j json -o dist
```

DICJ2A

This code reads JSON Dictionary and converts it to Archive Dictionaries.

Input file

- JSON Dictionary (`-i`)

Output files

- Archive Dictionaries (`-o`)

Arguments

- `-n` *`dict_ver`* dictionary version (transmission ID)
- `-i` *`dir_json`* directory of input JSON Dictionary
- `-o` *`dir_archive`* directory of output Archive Dictionaries

Example

Convert a JSON Dictionary *json/dict.json* to Archive Dictionaries *output/dict_arc.top, output/dict_arc_new.001* etc.

```
python3 x4_dicj2a.py -n 9131 -i json -o output
```

DICJ2T

This code reads JSON Dictionary and converts it to a Transmission (TRANS) Dictionary.

Input file

- JSON Dictionary (`-i`)

Output file

- TRANS Dictionary (`-o`)

Arguments

- `-n` *`dict_ver`* dictionary version (transmission ID)
- `-i` *`dir_json`* directory of input JSON Dictionary
- `-o` *`dir_trans`* directory of output TRANS Dictionary

Example

Convert JSON Dictionary *json/dict.json* to TRANS Dictionary *dist/trans.9131*.

```
python3 x4_dicj2t.py -n 9131 -i json -o dist
```

DIRINI

This code reads an EXFOR library tape (e.g., a master file[4]) in EXFOR formats with a `MASTER`, `LIB` or `REQUEST` record as the first record, splits it into entries, and saves each entry file in an entry storage. It initialises the storage (i.e., namely delete the files in the storage directory) at the beginning of processing.

Input file

- library tape (`-l`)

Output file

- directory of entry file storage (`-d`)
- log file (`-g`)

Arguments

- `-l` *`file_lib`* input library tape
- `-d` *`dir_storage`* directory of output entry storage
- `-g` *`file_log`* output log file (optional, default: "*x4_dirupd.log*")
- `-c` delete (1) trailing blanks in col. 12-66, (2) line sequential number (col.67-80) and (3) N2 of ENDBIB/ENDCOMMON/ENDSUBENT/ENDENTRY.

Example

Initialise the entry storage directory *entry* by loading the input library tape *library.txt* (without elimination of the record identification and bookkeeping if the input library has them) and record it in *x4_dirupd.log*:

```
python3 x4_dirini.py -l library.txt -d entry
```

At the end of processing, one obtains entry files under *entry*/1/, *entry*/2/, etc. and a log file with a new line like

```
Seq. Update date/time           Trans(N1) Trans(N2)  Centre Tape
   0 2023-10-04 00:48:30.849567 0001      20231004          library.txt
```

[4] The NRDC release the EXFOR Master File on an annual basis on the NRDC website (https://nds.iaea.org/nrdc/exfor-master/).

Note

A storage initialized by DIRINI with the most recent EXFOR Master File and updated by DIRUPD with all TRANS files released after the EXFOR Master File is equivalent to the zipped version of the EXFOR Entry Files distributed from the NRDC website (https://nds.iaea.org/nrdc/exfor-master/entry/).

DIRUPD

This code reads a trans tape[5] (starting from the `TRANS` record) and adds or updates the entry files in the local storage.

Input files

- trans tape (`-t`)
- directory of entry file storage (`-d`)
- log file (`-g`)

Output files

- entry files (`-d`)
- log file (`-g`)

Arguments

- `-t` *`file_trans`* input trans tape
- `-d` *`dir_storage`* (same as DIRINI)
- `-g` *`file_log`* input/output log file (optional, default: "*x4_dirupd.log*")
- `-c` (same as DIRINI. Use this option if you use it for DIRINI.)

Example

Update of the entry storage *entry* by a tape *trans.txt*, and record it in *x4_dirupd.log*:

```
python3 x4_dirupd.py -t trans.txt -d entry
```

At the end of processing, one obtains new and/or updated entry files under *entry*/1/ etc. and a log file with a new line like

```
Seq. Update date/time           Trans(N1) Trans(N2)  Centre Tape
   0 2023-10-04 00:48:30.849567 0001      20231004          library.txt
   1 2023-10-04 00:48:31.725707 1234      20160121   NNDC   trans.txt
```

[5] The NRDC assembles a set of new and revised EXFOR entries in a TRANS file on a regular basis, and it is released on the NRDC website (https://nds.iaea.org/nrdc/exfor-master/trans/)

EXTMUL

This code reads an EXFOR file containing subentries with the multiple reaction formalism, and write an extracted dataset as an EXFOR file. Three other Python scripts files for J4TOX4, POIPOI and X4TOJ4 must be placed together with EXTMUL in the same directory.

Input files

- EXFOR file (`-i`)
- JSON Dictionary (`-d`)

Output file

- EXFOR file (`-o`)

Arguments

- `-i` *`file_inp`* input EXFOR file
- `-d` *`file_dict`* input JSON Dictionary (optional, default: "*dict.json*")
- `-e` *`data_id`* EXFOR dataset ID for extraction (or "all" to extract all datasets)
- `-o` *`file_out`* output EXFOR file

Example

Extract a dataset 23756.002.3 from an EXFOR file *exfor.txt* to an EXFOR file *exfor_out.txt*:

```
python3 x4_extmul.py -i exfor.txt -e 23756.002.3 -o exfor_out.txt
```

Extract all datasets from an EXFOR file *exfor.txt* to an EXFOR file *exfor_out.txt*:

```
python3 x4_extmul.py -i exfor.txt -e all -o exfor_out.txt
```

Note that the first subentry (e.g., 23756.001) must present in the input EXFOR file.

J4TOC6

This code reads a J4 file and converts it to a C6 or C4 file. See Appendix 2 for the quantity and reaction specifications in these files.

Input file

- directory of J4 files processed by poipoi (`-i`)
- JSON Dictionary (`-d`)
- Heading mapping dictionary (*dict_arc_new.324*)
- Quantity mapping dictionary (*dict_arc_new.336*)

Output file

- C6 file (`-o`)
- index file (`-n`)

Arguments

- `-i` *`dir_j4`* input J4 file directory
- `-d` *`file_dict`* input JSON Dictionary (optional, default: "*dict.json*")
- `-e entry` EXFOR Entry number or "all" to process all J4 files in the input directory
- `-o` *`file_out`* output C6 or C4 file (or directory if separation level is 2 or 3)
- `-l seplvl` output separation level 1, 2 or 3 (optional, default: 3)[6]
- `-n file_idx` output index file (optional, default: "*x4_j4toc6.txt*")
- `-s sysout` output reference system identifier (optional, default: "OOO")[7]
- `-w enewid` energy width allowance (%) for SACS (optional, default: 10)[8]
- `-x` exclude heavy-ion (A>4) induced reactions
- `-y` include DERIVed data (`DERIV` in EXFOR REACTION SF9)
- `-c4` output in C4 format

[6] Level 1 prints all data points of the dataset in a single file. Level 2 is same as 1 but add blank lines at between two data blocks. Level 3 prints different blocks in different files. A **data block** is characterized by a set of common independent variable values. If dσ/dΩ dataset of elastic scattering (0 deg <θ< 180 deg) at 3 incident energies, the dataset consists of 3 data blocks and each block has the incident energy as the common independent variable.
[7] See LOTRAN for the reference system identifier.
[8] A spectrum averaged value (BRS, FIS, MXW, SDT, SPA) is converted if the lower and upper boundary energies are given and the ratio of their difference to the mean energies is within the energy width allowance.

Example

Read a J4 file processed by poipoi from the directory *j4* and write a C6 file directory *c6* for EXFOR entry 22742 with separation level 3:

```
python3 x4_j4toc6.py -i j4 -e 22742 -o c6
```

Same as above, but convert the reference frames of cross sections, outgoing angle and energy in those in centre-of-mass system:

```
python3 x4_j4toc6.py -i j4 -e 22742 -o c6 -s ccc
```

J4TOX4

This code reads a J4 file and converts it to an EXFOR file (starting from an `ENTRY`, `MASTER`, `REQUEST` or `TRANS` record).

Input files

- J4 file (`-i`)
- JSON Dictionary (`-d`)

Output file

- EXFOR file (`-o`)

Arguments

- `-i` *`file_j4`* input J4 file to read
- `-d` *`file_dict`* input JSON Dictionary (optional, default: "*dict.json*")
- `-o` *`file_x4`* output EXFOR file

Example

Convert a J4 file *exfor.json* to an EXFOR file *exfor.txt*:

```
python3 x4_j4tox4.py -i exfor.json -o exfor.txt
```

J4VIEW

This code reads J4 files processed by `POIPOI` with `-c` and convert it to an HTML file.

Input files

- directory of J4 files processed by POIPOI with `-c` (`-i`)
- JSON Dictionary (`-d`)

Output file

- HTML file (`-o`)

Arguments

- `-i` *`dir_j4`* directory of input J4 files processed by `POIPOI` to read
- `-d` *`file_dict`* input JSON Dictionary (optional, default: "*dict.json*")
- `-c` *`file_css`* input CSS file (optional, default: "*exfor.css*")
- `-e entry` EXFOR Entry number
- `-o` *`file_html`* output HTML file
- `-l` *`limit`* maximum number of data lines to print
- `-g` use sans-serif (gothic) font
- `-u` output with unformatted free text[9]

Example

Read J4 files processed by `POIPOI` with `-c` under the directory *j4*, and write an HTML file *exfor.html* for EXFOR entry E2789:

```
python3 x4_j4view.py -i j4 -e E2789 -o exfor.html
```

[9] See Appendix 1 for free text formatting.

LOTRAN

This code performs Lorentz transformation for (1) an angular differential cross section (ADX) of an exclusive channel (e.g., ^{56}Fe(p,n)^{56}Co) and (2) a double differential cross section (DDX) of an inclusive channel. (e.g., ^{56}Fe(p,n+x)).

Input files

- JSON Dictionary (`-d`)

Output file

- log file (`-g`)

Arguments

- `-d` *`file_dict`* input JSON Dictionary (optional, default: "*dict.json*")
- `-g` *`file_log`* output log file (optional, default: "*x4_lotran.log*")
- `-p id` datapoint identifier (optional, a positive integer, default "1")
- `-q` *`quan`* physical quantity (ADX or DDX)
- `-1` *`SF1`* target (e.g., 26-FE-56)
- `-2` *`SF2`* projectile (e.g., P)
- `-3` *`SF3`* ejectile (e.g., N)
- `-4` *`SF4`* product (e.g., 27-CO-56)
- `-t` *`tinc`* laboratory incident kinetic energy (eV)
- `-a` *`ang3`* ejectile angle (deg)
- `-A` d*`ang3`* ejectile angle uncertainty (deg) (optional, default: 0)
- `-e` *`e3`* ejectile kinetic energy (eV) for DDX
- `-E` de*`3`* ejectile kinetic energy uncertainty (eV) for DDX (optional, default: 0)
- `-c` *`sig`* cross section (b/sr or b/sr/eV)
- `-C` *`sig`* cross section uncertainty (b/sr or b/sr/eV) (optional, default: 0)
- `-x` *`eexc`* residual excitation energy (eV)
- `-s` *`sysinp`* input reference system identifier
- `-S` *`sysout`* output reference system identifier

The reference system identifier is a set two characters for ADX and three characters for DDX. The first character is for the reference frame of the cross section, second character is for the reference frame of the ejectile angle, and the third character is the reference frame of the ejectile kinetic energy. Each character is L (laboratory system) or C (centre-of-mass system). For example, LL means the angular differential cross section in the laboratory system given with the ejectile angle in the laboratory system, and CCL means the double differential cross section in the centre-of-mass system given with the ejectile angle in the centre-of-mass system and ejectile kinetic energy in the laboratory system.

Summary of reference system identifiers

Identifier	dσ/dΩ	θ	Identifier	dσ/dΩdE	θ	E
LL	lab.	lab.	LLL	lab.	lab.	lab.
LC	lab.	cm	LLC	lab.	lab.	c.m.
CL	cm.	lab.	LCL	lab.	c.m.	lab.
CC	cm.	cm.	LCC	lab.	c.m.	c.m.
			CLL	c.m.	lab.	lab.
			CLC	c.m.	lab.	c.m.
			CCL	c.m.	c.m.	lab.
			CCC	c.m.	c.m.	c.m.

Example

Convert dσ/dΩcm=1.666±0.030 mb/sr at θcm=35.21 deg for ^{208}Pb(d,p)^{209}Pb (Ex=2.032 MeV) at 22 MeV to dσ/dΩlab and θlab):

```
python3 -P E1705.007 -q adx -1 82-PB-208 -2 D -3 P -4 82-PB-209
-t 22E+06 -a 35.21 -c 1.666E-03 -C 0.030E-03 -x 2.032E+06 -s CC -S LL
```

It prints the result in the following log file (x4_lotran.log):

```
----------------------------------------------------------------------
LOTRAN report 2026-09-26 23:28:08
----------------------------------------------------------------------
Target                                          : 82-PB-208
Projectile                                      : 1-H-2
Ejectile                                        : 1-H-1
Residual                                        : 82-PB-209
Laboratory incident energy (eV)                 : 22000000.0
Residual excitation energy (eV)                 : 2032000.0
Input ejectile angle reference system           : c.m.
Input cross section reference system            : c.m.
Input ejectile angle (deg)                      : 35.21
Input cross section (b/sr)                      : 0.001666
Input cross section uncertainty (b/sr)          : 3e-05
Output ejectile angle reference system          : lab.
Output cross section reference system           : lab.
Output ejectile angle (deg)                     : 34.979
Output cross section (b/sr)                     : 1.68521E-03
Output cross section uncertainty (b/sr)         : 3.03460E-05
----------------------------------------------------------------------
```

MAKC6L

This code reads the J4 files in a J4 file storage generated by MAKJ4L, and convert them to C6 or C4 files in a C6/C4 file storage.

Input file

- directory of J4 file storage (`-i`)
- JSON Dictionary (`-d`)
- Heading mapping dictionary (*dict_arc_new.324*)
- Quantity mapping dictionary (*dict_arc_new.336*)

Output file

- directory of C6/C4 file storage (`-o`).

Arguments

- `-i` *`dir_j4lib`* input J4 storage directory
- `-d` *`file_dict`* input JSON Dictionary (optional, default: "*dict.json*")
- `-o` *`dir_c6lib`* directory of output C6/C4 file storage
- `-l` *`seplvl`* output separation level 1, 2 or 3 (optional, default: 3)[10]
- `-t` *`file_trans`* trans tape containing entries for C6/C4 file production (optional)
- `-g` *`file_log`* output log file (optional, default: "*x4_makc6l.log*")
- `-n` *`file_idx`* output index file (optional, default: "*x4_makc6l.txt*")
- `-s` *`sysout`* output reference system identifiers (optional, default: "OOO")[11]
- `-w` *`enewid`* energy width allowance (%) for SACS (optional, default: 10)[12]
- `-x` exclude heavy-ion (A>4) induced reactions
- `-y` include DERIVed data (`DERIV` in EXFOR REACTION SF9)
- `-c4` output in C4 format

[10] See MAKC6L for the separation level.
[11] See LOTRAN for the reference system identifier.
[12] See J4TOC6 for the energy width allowance.

Example

Create C6 files under the storage *c6* by converting the J4 files in the storage *j4*:

```
python3 x4_makc6l.py -i j4 -o c6
```

When a trans tape is not specified and the specified directory of the C6 file storage exists, all files under the C6 file storage are deleted before starting conversion of the J4 files to C6 files.

MAKCOV

This code reads a J4 file processed by `POIPOI`, and print a four-column (x, Δx, y, Δy) table and correlation coefficients (upper triangle matrix).

Input files

- J4 file processed by `POIPOI` with `-k all` and without `-p` and `-c` (`-i`)
- "HED file" (`-j`)[13]
- JSON Dictionary (`-d`)

Output files

- covariance file (`-o`)
- log file (`-g`)

Arguments

- `-i` *`file_j4`* input J4 file
- `-j` *`file_hed`* input "HED file"
- `-d` *`file_dict`* input JSON Dictionary (optional, default: "*dict.json*")
- `-e` *`data_id`* EXFOR Dataset ID[14]
- `-o` *`file_cov`* output covariance file
- `-g` *`file_log`* output log file (optional, default: "*x4_makcov.log*")
- `-l` *`x_min`* lower boundary of independent variable (optional)
- `-u` *`x_max`* upper boundary of independent variable (optional)
- `-s` treat the dataset as a shape (normalization free) dataset
- `-r` print Δy/y (%) rather than Δy

[13] See Appendix 4 for the details of the HED file.
[14] To identify the corresponding dataset ID (e.g., 22742.004.1) in the J4 file, the J4 file must be created by `POIPOI` without the option -d (delete pointer).

Example

Read a J4 file *exfor.json*, HED file *exfor_hed.txt*, and create a table file *x4_makcov.txt* for a dataset *22742.004.1*:

```
python3 x4_makcov.py -i exfor.json -j exfor_hed.txt -e 22742.004.1 -o
x4_makcov.txt
```

MAKCXL

This code reads the C6 files in a C6 file storage generated by MAKC6L, and convert them to CX4 files in a CX4 file storage.

Input file

- directory of C6 file storage (`-i`)
- JSON Dictionary (`-d`)
- Heading mapping dictionary (*dict_arc_new.324*)
- Quantity mapping dictionary (*dict_arc_new.336*)
- gnuplot command setup file (*gnuplot_command.txt*) if `-p` option is activated.[15]

Output file

- directory of CX4 file storage (`-o`).

Arguments

- `-i` *`dir_c6lib`* input C6 storage directory
- `-d` *`file_dict`* input JSON Dictionary (optional, default: "*dict.json*")
- `-o` *`dir_cx4lib`* directory of output CX4 file storage
- `-t` *`file_trans`* trans tape containing entries for CX4 file production (optional,)
- `-g` *`file_log`* output log file (optional, default: "*x4_makcxl.log*")
- `-n` *`file_idx`* output index file (optional, default: "*x4_makcxl.txt*")
- `-s` *`sysout`* output reference system identifiers (optional, default: "OOO")[16]
- `-p` plot the CX4 file by gnuplot
- `-x` exclude heavy-ion (A>4) induced reactions

Example

Create CX4 files under the storage *cx4* by converting the C6 files in the storage *c6*:

```
python3 x4_makcxl.py -i c6 -o cx4
```

[15] See PLOTCX for the contents of the setup file.
[16] See LOTRAN for the reference system identifier.

When a trans tape is not specified and the specified directory of the CX4 file storage exists, all files under the CX4 file storage are deleted before starting conversion of the C6 files to CX4 files.

MAKJ4L

This code reads the entry files in an entry storage generated by DIRINI or DIRUPD, and convert them to J4 files in a J4 file storage.

Input file

- directory of entry file storage (`-i`)
- JSON Dictionary (`-d`)

Output file

- directory of J4 file storage (`-j`).

Arguments

- `-i` *`dir_storage`* (same as DIRINI)
- `-d` *`file_dict`* input JSON Dictionary (optional, default: "*dict.json*")
- `-j` *`dir_j4lib`* directory of output J4 file storage
- `-t` *`file_trans`* trans tape containing entries for J4 file production (optional)
- `-g` *`file_log`* output log file (optional, default: "*x4_makj4l.log*")
- `-m email` your email address for addition of DOI to J4 (optional)

Example

Create J4 files under the storage *j4* by converting the entry files in the storage *entry*:

```
python3 x4_makj4l.py -i entry -j j4
```

Same as above, but do conversion only for entries included in *trans.txt*:

```
python3 x4_makj4l.py -i entry -t trans.txt -j j4
```

Note: When a trans tape is not specified and the specified directory of the J4 file storage exists, all files under the J4 file storage are deleted before starting conversion of the entry files to J4 files.

MAKLIB

This code reads and combines the entry files in the entry storage and create a single library tape.

Input file

- Directory of entry file storage (`-d`)

Output file

- library tape (`-l`).

Arguments

- `-d` *`dir_storage`* (same as DIRINI)
- `-l` *`file_lib`* output library tape
- `-i` *`tape_id`* Tape ID (integer printed at cols 12-22 of the first record)
- `-a` add "19" to two-digit year in N2 of ENTRY/SUBENT/NOSUBENT.
- `-c` (same as DIRINI.)
- `-n` exclude dictionaries

Example

Create a library tape *library.txt* by merging entry files in the storage *entry* with the tape ID *0001*:

```
python3 x4_maklib.py -d entry -l library.txt -i 0001
```

This operation does not add a new line in the log file.

Note: The format of the output library tape depends on the format of the files in the entry storage. If user maintains the entry files in the storage without record identifications etc. (e.g., with -c option of DIRINI and DIRUPD), then the produced library tape also does not have them. The record identifications can be added later by processing the output library tape by SEQADD.

PLOTCX

This code plots CX4 file by using gnuplot. Execution requires installation of **gnuplot**,

Input file

- Input CX4 file (`-i`)
- gnuplot command setup file (*gnuplot_command.txt*)

  Example of the gnuplot command setup file

```
# Example for Linux
/usr/bin/gnuplot
#
# Example for Windows
#C:\Programs\gnuplot\bin\gnuplot.exe
```

Output file

- Output pdf file (`-o`).

Arguments

- `-i` *`file_cx4`* input CX4 file following standard CX4 file naming convention[17]
- `-o` *`file_pdf`* output pdf file

Example

Plot Ti+n total cross sections for monoenergetic beam measured by Schwartz (1974) and compiled in EXFOR 10003.002:

```
python3 x4_plotcx.py

-i n-Ti-tot-sig-excfun-mon-Schwartz_1974_10003.002.cx4

-o n-Ti-tot-sig-excfun-mon-Schwartz_1974_10003.002.pdf
```

[17] See Appendix 3 for the standard file naming convention.

POIPOI

This code reads a J4 file, extract the information relevant to a particular dataset, and create a J4 file. It removes the pointer structure in the input J4 file. If wish, one can (1) merge the information originally compiled in the common (001) subentry and the data subentry, and (2) select the keywords (e.g., TITLE, AUTHOR) to be kept in the output J4 file.

Input files

- J4 file (`-i`)
- JSON Dictionary (`-d`)

Output file

- J4 file (`-o`)

Arguments

- `-i` *`file_inp`* input J4 file
- `-d` *`file_dict`* input JSON Dictionary (optional, default: "*dict.json*")
- `-e` *`data_id`* EXFOR Dataset ID[18], EXFOR entry number, or "all" to process all datasets in the input)
- `-o` *`file_out`* output J4 directory (or file if EXFOR Dataset ID is given)
- `-k @`*`key_keep`* BIB keywords to keep (optional, default: "all")[19]
- `-p` delete the pointer
- `-c` keep the common (001) and data subentries separately
- `-r` keep resonance parameter table[20]

Example

Read a J4 file *exfor.json*, and create another J4 file *exfor_poi.json* for a dataset *22742.004.1*: by keeping the BIB records under AUTHOR and REFERENCE:

```
python3 x4_poipoi.py -i exfor.json -e 22742.004.1 -o exfor_poi.json
```

Same as above, but remove all data descriptions other than those under AUTHOR and REFERENCE:

[18] For example, 22742.004 or 22742.004.1.
[19] The REACTION information is always kept.
[20] With this option, resonance parameters under the pointers are kept in the same subentry without decomposition.

```
python3 x4_poipoi.py -i exfor.json -e 22742.004.1 -o exfor_poi.json
-k AUTHOR REFERENCE
```

REFBIB

This code converts an EXFOR reference code to DOI and vice versa. Then it extracts various bibliography related items (e.g., title, authors) from the metadata deposited in DOI registration agencies, and outputs them in various format. Currently this code supports only DOIs of journal articles registered in CrossRef.

Execution requires an active internet connection as well as Python modules **pylatexenc**, **pyspellchecker** and **requests**, which may be installed by the following command if pip (a standard Python package manager) is installed in your computer:

```
pip install pylatexenc

pip install pyspellchecker

pip install requests
```

Input file

- JSON Dictionary (`-d`)

Output file

- Bibliography file (`-o`)

Arguments

- `-i` *`ref_inp`* EXFOR reference code or DOI
- `-a` *`fauthor`* Family name of the first author (optional, default: "any")
- `-d` *`file_dict`* input JSON Dictionary
- `-o` *`file_bib`* output bibliography file
- `-r` *`form`* output format (doi, piped, exfor, bibtex, json or xml)
- `-m` *`email`* your email address (for notification to CrossRef)
- `-s` strip accent symbols in output

Example

For the EXFOR reference code *J,NDS,120,272,2014* and email address *email@address.com*, write DOI in *x4_refbib.txt*:

```
python3 x4_refbib.py -i J,NDS,120,272,2014 -o x4_refbib.txt -r doi -m
email@address.com
```

REFDOI

This code reads an EXFOR file, and check presence of a DOI for each article coded under REFERENCE, REL-REF, MONIT-REF and STATUS when the article is from a journal volume registered in CrossRef. Three Python scripts files for REFCHK, X4TOJ4 and REFBIB must be placed in the same directory.

Execution of REFDOI requires an active internet connection as well as Python modules **pylatexenc**, **pyspellchecker** and **requests**. See the page for REFDOI for their installation.

Input files

- EXFOR file (`-i`)
- JSON Dictionary (`-d`)
- Email address (`-m`)

Output file

- DOI file (`-o`)

Arguments

- `-i` *`file_x4`* input EXFOR file
- `-d` *`file_dict`* input JSON Dictionary (optional, default: "*dict.json*")
- `-o` *`file_doi`* output DOI file
- `-j` *`file_json`* output JSON file
- `-k` `@`*`key_anal`* keywords for processing (optional, default: "REFERENCE")
- `-m` *`email`* your email address (for notification to CrossRef)

Note that the DOI file is overwritten while JSON file is updated if exists.

Example

Check reference codes under REFERENCE included in *exfor.txt*:

```
python3 x4_refdoi.py -i exfor.txt -o x4_refdoi.txt -j x4_refdoi.json
-m email@address.com
```

Same as above, analyse codes under REFERENCE and STATUS:

```
python3 x4_refdoi.py -i exfor.txt -o x4_refdoi.txt -j x4_refdoi.json
-k REFERENCE STATUS -m email@address.com
```

SEQADD

This code adds and/or updates record identifications (cols.67-79) and bookkeeping information such as N1 and N2 of BIB and ENDBIB. records. (Similar to ORDER developed at NNDC and ZORDER developed at NDS).

Input file

- Entry, trans or library tape (`-i`)

Output file

- Entry, trans or library tape (`-o`)

Arguments

- `-i` *`file_inp`* input EXFOR file
- `-o` *`file_out`* output EXFOR file
- `-m` do not add "19" to two-digit year in N2 of ENTRY/SUBENT/NOSUBENT, and do not alter N2 of ENDBIB/ENDCOMMON/ENDDATA/ENDSUBENT/ENDENTRY/ENDSUBDICT records.
- `-d` delete the transmission ID in N6

Example

Process an EXFOR file *exfor.txt* by adding and updating the record identification and bookkeeping information, and print the output in *exfor_ord.txt*.

```
python3 x4_seqadd.py -i exfor.txt -o exfor_ord.txt
```

SPELLS

This code checks English spells in the free text field in the EXFOR format. It checks each set of lower characters in free text (i.e., the first word of a sentence is not checked). The default dictionary does not know nuclear physics technical terms, and the user should add them to the dictionary to minimize the output.

Execution requires a Python module **pyspellchecker**, which may be installed by the following command if pip (a standard Python package manager) is installed in your computer:

```
pip install pyspellchecker
```

Input files

- Entry, trans or library tape (`-i`)
- Dictionary listing known technical words (`-d`)

  Example of the dictionary file (a plain text file to be updated by the user by adding more technical terms etc.)

```
atm
deadtime
decoupler
epithermal
fluence
linac
nonuniformity
subentry
```

Output file

- list of typos (`-l`)

Arguments

- `-i` *`file_x4`* input EXFOR file
- `-d` *`file_dict`* input known word dictionary (optional, default: "*x4_spells.dic*")
- `-o` *`file_typo`* output typo list file

Example

Check spells in an EXFOR entry file *exfor.txt,* and record the checking result in *x4_spells_out.txt*.

```
python3 x4_spells.py -i exfor.txt -o x4_spells_out.txt
```

X4TOC6

This code reads an EXFOR file and converts it to a C6 or C4 file.

Input file

- EXFOR file (`-i`)
- JSON Dictionary (`-d`)
- Heading mapping dictionary (*dict_arc_new.324*)
- Quantity mapping dictionary (*dict_arc_new.336*)

Output file

- C6 file (`-o`)
- index file (`-n`)

Arguments

- `-i` *`file_x4`* input EXFOR file
- `-d` *`file_dict`* input JSON Dictionary (optional, default: "*dict.json*")
- `-e entry` EXFOR Entry number
- `-o` *`file_out`* output C6 or C4 file (or directory if separation level is 2 or 3)
- `-l seplvl` output separation level 1, 2 or 3 (optional, default: 3)[21]
- `-n file_idx` output index file (optional, default: "*x4_makc6l.txt*")
- `-s sysout` output reference system identifier (optional, default: "OOO")[22]
- `-w enewid` energy width allowance (%) for SACS (optional, default: 10%)[23]
- `-m email` your email address for addition of DOI to J4 (optional)
- `-x` exclude heavy-ion (A>4) induced reactions
- `-y` include DERIVed data (`DERIV` in EXFOR REACTION SF9)
- `-c4` output in C4 format

[21] See J4TOC6 for the separation level.
[22] See LOTRAN for the reference system identifier.
[23] See J4TOC6 for the energy width allowance.

Example

Read an EXFOR file *exfor.txt,* and write C6 files in directory *c6* for EXFOR entry 22742:

```
python3 x4_j4toc6.py -i exfor.txt -e 22742 -o c6
```

X4TOCX

This code reads an EXFOR file and converts it to CX4 files.

Input files

- EXFOR file (`-i`)
- JSON Dictionary (`-d`)
- Heading mapping dictionary (*dict_arc_new.324*)
- Quantity mapping dictionary (*dict_arc_new.336*)
- gnuplot command setup file (*gnuplot_command.txt*) if `-p` option is activated.[24]

Output file

- CX4 file (`-o`)
- Index file (`-n`)

Arguments

- `-i` *`file_x4`* input EXFOR file
- `-d` *`file_dict`* input JSON Dictionary (optional, default: "*dict.json*")
- `-e entry` EXFOR Entry number or "all" to process all C6 files in the input directory
- `-o` *`dir_cx4`* output CX4 file directory
- `-n` *`file_idx`* output index file (optional, default: "*x4_x4tocx.txt*")
- `-s` *`sysout`* output reference system identifier (optional, default: "OOO")[25]
- `-w enewid` energy width allowance (%) for SACS (optional, default: 10)[26]
- `-m email` your email address for addition of DOI to J4 (optional)
- `-p` plot the CX4 file by gnuplot (If a Python module **pypdf** is installed, pdf files are merged into a pdf file.)
- `-x` exclude heavy-ion (A>4) induced reactions
- `-y` include DERIVed data (`DERIV` in EXFOR REACTION SF9)

[24] See PLOTCX for the contents of the setup file.
[25] See LOTRAN for the reference system identifier.
[26] See J4TOC6 for the energy width allowance.

Example

Read an EXFOR file *exfor.txt,* and write CX4 files in directory *cx4* for EXFOR entry 22742:

```
python3 x4_x4tocx.py -i exfor.txt -e 22742 -o cx4
```

X4TOJ4

This code reads an EXFOR file (starting from an `ENTRY`, `MASTER`, `REQUEST` or `TRANS` record) and convert it to a J4 file. One can select the keywords (e.g., TITLE, AUTHOR) to be kept in the J4 file.

Input files

- EXFOR file (`-i`)
- JSON Dictionary (`-d`)

Output file

- J4 file (`-o`)

Arguments

- `-i` *`file_x4`* input EXFOR file
- `-d` *`file_dict`* input JSON Dictionary (optional, default: "*dict.json*")
- `-e data_id` EXFOR Entry number, or "all" to process all entries in the input file
- `-o` *`file_j4`* output J4 file
- `-k @`*`key_keep`* BIB keywords to be kept (optional, default: "all")[27]
- `-m email` your email address for addition of DOI to J4 (optional)
- `-c` check record identification of the system identifiers
- `-a` add "19" to two-digit year
- `-g` keep the flag at column 80 of each record of the DATA sections
- `-s` keep real numbers as strings

Note that this code sets the integers in the BIB(N2), ENDBIB(N1), COMMON(N2), ENDCOMMON(N1), ENDDATA(N1) and ENDSUBENT (N1) of the J4 file to 0.

Example

Read an EXFOR file *exfor.txt*, and write a J4 file *exfor.json*:

```
python3 x4_x4toj4.py -i exfor.txt -o exfor.json
```

Same as above, but keep descriptions under AUTHOR, REFERENCE and REACTION only:

[27] The REACTION information is always kept.

```
python3 x4_x4toj4.py -i exfor.txt -o exfor.json -k AUTHOR REFERENCE
REACTION
```

X4VIEW

This code reads an EXFOR file, process it by SEQADD, and convert it to an HTML file. Three other Python scripts files for J4VIEW, POIPOI, SEQADD and X4TOJ4 must be placed together with X4VIEW in the same directory.

Input files

- EXFOR file (`-i`)
- JSON Dictionary (`-d`)

Output file

- HTML file (`-o`)

Arguments

- `-i` *`file_x4`* input EXFOR file
- `-d` *`file_dict`* input JSON Dictionary (optional, default: "*dict.json*")
- `-c` *`file_css`* input CSS file (optional, default: "*exfor.css*")
- `-e entry` EXFOR Entry number
- `-o` *`file_html`* output HTML file
- `-l` *`limit`* maximum number of data lines to print
- `-m` *`email`* your email address for addition of DOI (optional)
- `-g` use sans-serif (gothic) font
- `-u` output with unformatted free text[28]

Example

Read an EXFOR file *exfor.txt*, and write an HTML file *exfor.html* for EXFOR entry E2789:

```
python3 x4_x4view.py -i exfor.txt -e E2789 -o exfor.html
```

[28] See Appendix 1 for free text formatting.

Appendix 1: Output of formatted free text to HTML by J4VIEW / X4VIEW

By running J4VIEW or X4VIEW without the unformatted free text output option (-u), one can introduce some structures to the free text field of the input EXFOR entry. Currently, paragraph, heading, itemizing and table are supported.

1. Paragraph

   Presence of an alphabetical character or number at column 12 indicates opening of a new paragraph, and the free text is indented.

2. Heading

   Presence of an asterisk at column 12 indicates a heading line, and the free text is not indented.

3. Itemizing

   Presence of a minus (−) or full stop (.) character at column 12 followed by a blank indicates an item, and the free text follows a bullet.

4. Table

   Presence of a plus character followed by two minus character (+−−) indicates beginning or ending of a table in free text.

Example:

Input

```
SAMPLE     Metallic foils of Pt, Ti and Al used:                E281100100011
             +---------------------------------------+          E281100100012
                                Pt     Ti     Al               E281100100013
             ----------------------------------------          E281100100014
              Nominal purity(%)  99.95   99.6   >99            E281100100015
             ----------------------------------------          E281100100016
              Thickness (mg/cm2) 21.672  2.343  1.817          E281100100017
             +---------------------------------------+          E281100100018
METHOD     Foils were                                           E281100100019
            - irradiated for 52 min at ~204 nA                  E281100100020
            - cooled for 54 min                                 E281100100021
COMMENT    * By Naohiko Otuka (2025-08-10):                     E281100100022
           This work was carried out at RI Beam Factory operated E281100100023
            by RIKEN Nishina Center and CNS, University of Tokyo, E281100100024
            Japan.                                              E281100100025
           The authors would like to thank Dr. Y. Shigekawa, Dr.  E281100100026
            Y. Kanayama, and Dr. H. Shimizu (RIKEN Nishina Center)E281100100027
            for technical assistance with the experiment.       E281100100028
```

Output

Sample

Metallic foils of Pt, Ti and Al used:

	Pt	Ti	Al
Nominal purity(%)	99.95	99.6	>99
Thickness (mg/cm2)	21.672	2.343	1.817

Method

Foils were

- irradiated for 52 min at ~204 nA
- cooled for 54 min

Comment

* By Naohiko Otuka (2025-08-10):

This work was carried out at RI Beam Factory operated by RIKEN Nishina Center and CNS, University of Tokyo, Japan.

The authors would like to thank Dr. Y. Shigekawa, Dr. Y. Kanayama, and Dr. H. Shimizu (RIKEN Nishina Center) for technical assistance with the experiment.

Appendix 2: Quantity and reaction specifications in C4 and C6 formats

In C4 and C6 formats, the quantity and reaction of the dataset are specified by the MT and MF numbers defined, which are similar to those defined for the ENDF format. The next tables summarize the relation ship of the (MF,MT) numbering and EXFOR REACTION coding (SF2 to SF7).

Definition of MF number

MF	SF6	SF7	Quantity
1	NU		Fission neutron multiplicity
3	SIG		Cross section
4	DA		Angular differential cross section
5	DE		Energy differential cross section
6	DA/DE		Double differential cross section
8	FY		Fission product yield
13	SIG	G	Gamma production cross section
14	DA	G	Gamma production angular differential cross section
15	DE	G	Gamma production energy differential cross section

Definition of MT number

- *: Any code (but SF3 must be one of SIG, DA, DE or DA/DE)
- . N in SF3: SF3 must be INL when SF2=N
- + in SF3: SF3=X or F
- + in SF5: SF5= brank or IND
- + in SF6: SF6= SIG or FY
- + in SF7: SF7=brank or G with SF6=SIG, DA or DE

MT	SF2	SF3	SF4	SF5	SF6	SF7
1	*	TOT			*	
2	*	EL	*		*	
3	*	NON			*	
4	*	. N	*		*	+
11	*	2N+D	*		SIG	+
16	*	2N	*		SIG	+
17	*	3N	*		SIG	+
18	*	F			SIG	
22	*	N+A	*		SIG	+
23	*	N+3A	*		SIG	+
24	*	2N+A	*		SIG	+
25	*	3N+A	*		SIG	+
27	*	ABS	*		SIG	
28	*	N+P	*		SIG	
29	*	N+2A	*		SIG	+
30	*	2N+2A	*		SIG	+
32	*	N+D	*		SIG	+

33	*	N+T	*			SIG	+
34	*	N+HE3	*			SIG	+
35	*	N+D+2A	*			SIG	+
36	*	N+T+2A	*			SIG	+
37	*	4N	*			SIG	+
41	*	2N+P	*			SIG	
42	*	3N+P	*			SIG	
44	*	N+2P	*			SIG	+
45	*	N+P+A	*			SIG	+
50	*	N	*		PAR	*	+
51	N	INL	*		PAR	*	+
102	*	. G	*			*	+
103	*	. P	*			*	+
104	*	. D	*			*	+
105	*	. T	*			*	+
106	*	. HE3	*			*	+
107	*	. A	*			*	+
108	*	2A	*			SIG	+
109	*	3A	*			SIG	+
111	*	2P	*			SIG	+
112	*	P+A	*			SIG	+
113	*	T+2A	*			SIG	+
114	*	D+2A	*			SIG	+
115	*	P+D	*			SIG	+
116	*	P+T	*			SIG	+
117	*	D+A	*			SIG	+
152	*	5N	*			SIG	+
153	*	6N	*			SIG	+
154	*	2N+T	*			SIG	+
155	*	T+A	*			SIG	+
156	*	4N+P	*			SIG	
157	*	3N+D	*			SIG	+
158	*	N+D+A	*			SIG	+
159	*	2N+P+A	*			SIG	
160	*	7N	*			SIG	+
161	*	8N	*			SIG	+
162	*	5N+P	*			SIG	
163	*	6N+P	*			SIG	
164	*	7N+P	*			SIG	
165	*	4N+A	*			SIG	+
166	*	5N+A	*			SIG	+
167	*	6N+A	*			SIG	+
168	*	7N+A	*			SIG	+
169	*	4N+D	*			SIG	
170	*	5N+D	*			SIG	
171	*	6N+D	*			SIG	

172	*	3N+T	*		SIG	+
173	*	4N+T	*		SIG	+
174	*	5N+T	*		SIG	+
175	*	6N+T	*		SIG	+
176	*	2N+HE3	*		SIG	+
177	*	3N+HE3	*		SIG	+
178	*	4N+HE3	*		SIG	+
179	*	3N+2P	*		SIG	
180	*	3N+2A	*		SIG	+
181	*	3N+P+A	*		SIG	
182	*	D+T	*		SIG	+
183	*	N+P+D	*		SIG	
184	*	N+P+T	*		SIG	
185	*	N+D+T	*		SIG	+
186	*	N+P+HE3	*		SIG	
187	*	N+D+HE3	*		SIG	+
188	*	N+T+HE3	*		SIG	+
189	*	N+T+A	*		SIG	+
190	*	2N+2P	*		SIG	
191	*	P+HE3	*		SIG	+
192	*	D+HE3	*		SIG	+
193	*	HE3+A	*		SIG	+
194	*	4N+2P	*		SIG	
195	*	4N+2A	*		SIG	+
196	*	4N+P+A	*		SIG	
197	*	3P	*		SIG	+
198	*	N+3P	*		SIG	
199	*	3N+2P+A	*		SIG	
200	*	5N+2P	*		SIG	
201	*	X	0-NN-1		*	
202	*	X	0-G-0		*	
203	*	X	1-H-1		*	
204	*	X	1-H-2		*	
205	*	X	1-H-3		*	
206	*	X	2-HE-3		*	
207	*	X	2-HE-4		*	
208	*	X	1-PP-0		*	
209	*	X	0-P0-0		*	
210	*	X	1-PN-0		*	
213	*	X	1-KP-0		*	
216	*	X	1-KN-0		*	
217	*	X	1-AP-1		*	
452	*	F			NU	
454	*	+	ELEM/MASS	+	+	
455	*	F		DL	NU	
456	*	F		PR	NU	

459	*	+	ELEM/MASS	CUM	+	
218	*	X	0-AN-1		*	
600	*	P	*	PAR	*	+
601	P	INL	*	PAR	*	+
650	*	D	*	PAR	*	+
651	D	INL	*	PAR	*	+
700	*	T	*	PAR	*	+
701	T	INL	*	PAR	*	+
750	*	HE3	*	PAR	*	+
751	HE3	INL	*	PAR	*	+
800	*	A	*	PAR	*	+
801	A	INL	*	PAR	*	+

Appendix 3: Standard file naming convention

A file name following the standard file naming convention includes various fields characterizing the contents of the file such as the reaction and quantity.

C6/C4 file: $did-$proj-$targ-$proc-$quant-$dist-$spec-$author_$year.c6 or .c4

CX4 file: $proj-$targ-$proc-$quant-$dist-$spec-$consts_$author_$year_$did.cx4

did	dataid	EXFOR data ID (e.g., 12345.012, 12345.012.1)
proj	projectile	particle or nuclide symbol (e.g., g, n, p, d, t, h, a, Fe56)
targ	target	Nuclide symbol (e.g., Fe56, Fe)
proc	process	process or particle symbol (e.g., tot, abs, non, f, 0, x, g, n, p, d, t, h, a, Fe56)
quant	quantity	• adxc: angular differential cross section in c.m. system • adxl: angular differential cross section in lab. system • ddxc: double differential cross section in c.m. system • ddxl: double differential cross section in lab. system • edxc: energy differential cross seciton in c.m. system • edxl: energy differential cross seciton in lab. system • sig: cross section • sigc: cross section (cumulative) • fyc: fission product yield (cumulative) • fyi: fission product yield (independent) • nud: fission neutron multiplicity (delayed) • nup: fission neutron multiplicity (prompt) • nut: fission neutron multiplicity (total)
dist	distribution	• angdisc: c.m. angle dependence • angdisl: lab. angle dependence • enedisc: c.m. outgoing energy dependence • enedisl: lab. outgoing energy dependence • excfun: incident energy dependence • lvldis: level energy dependence • nucdis: product nuclide dependence • othdis: other distribution (probably indication of processing error!) • sponnu: spontaneous fission neutron multiplicity
spec	spectrum	• brs: bremsstrahlung spectrum average and unfolded • fis: fission neutron spectrum field • mon: mono energetic field • mxw: Maxwellian field • sdt: slow-down time spectrometer field • spa: averaged over a spectrum with ~0.0253 eV or narrow energy width
consts	constant	• angc: c.m. angle • angl: lab. angle

		• elvl: excitation energy (of product) • enec: c.m. outgoing energy • enel: lab. outgoing energy • prod: product
author	author	First author's name. A blank and "-" in author's name are replaced by "%" and "+" in the file name (e.g., "Abd%El+Kariem" for "Abd El-Kariem".)
year	year	four-digits year

Example of C6/C4 file names

d4408.005-a-Ni-x-sig-excfun-mon-Takacs_2020.c4

Ni(a,x) production cross section as a function of the incident energy (excfun), published by Takacs in 2020 and compiled in EXFOR D4408.005.

10037.024-n-Fe56-el-adxc-angdisc-mon-Boschung_1971.c4

^{56}Fe(n,n$_0$)^{56}Fe angular differential cross sections in the c.m. system (adxc) as a function of the c.m. angle (angdisc), published by Boschung in 1971 and compiled in EXFOR 10037.024.

c0841.008-p-Bi209-xn-ddxl-enedisl-mon-Kalend_1983.c4

^{209}Bi(p,n+x) double differential cross section in the lab. system (ddxl) as a function of the outgoing neutron energy in the lab. system (enedisl), published by Kalend in 1983 and compiled in EXFOR C0841.008.

Examples of CX4 file names

a-Ni-x-sig-excfun-mon-prodCu64_Takacs_2020_D4408.005.cx4

Ni(a,x)^{64}Cu production cross section as a function of the incident energy (excfun), published by Takacs in 2020 and compiled in EXFOR D4408.005.

n-Fe56-el-adxc-angdisc-mon-einc5.050E+06_prodFe56_Boschung_1971_10037.024.cx4

^{56}Fe(n,n$_0$)^{56}Fe angular differential cross sections in the c.m. system (adxc) at the incident energy (einc) of 5.05 MeV as a function of the c.m. angle (angdisc), published by Boschung in 1971 and compiled in EXFOR 10037.024.

p-Bi209-xn-ddxl-enedisl-mon-einc9.000E+07_angl20_Kalend_1983_C0841.008.cx4

^{209}Bi(p,n+x) double differential cross section in the lab. system (ddxl) at the incident energy (einc) of 90 MeV and outgoing angle in the lab. system (angl) at 20 deg as a function of the outgoing neutron energy in the lab. system (enedisl), published by Kalend in 1983 and compiled in EXFOR C0841.008.

Appendix 4: HED file used in MAKCOV

(See also N. Otuka, O. Iwamoto, INDC(SEC)-0112(Rev.) Sect.2.3)

Basic structure

The HED file defines the independent (x) and dependent (y) variables for tabulation by MAKCOV. It may have the following lines:

- L.1: Heading for the lower boundary of the independent variable.
- L.2: Heading for the upper boundary of the independent variable (may be repetition of the heading on L.1).
- L.3: Heading for the uncertainty or resolution of the independent variable (optional). The number under this heading is ignored if it is smaller than the difference of the numbers under the headings specified in L.1 and L.2.
- L.4: Heading for the dependent variable. The heading specified in L.4 is typically `DATA` or `DATA-CM`.
- L.5: Heading for the reference variable (optional). The heading specified in L.5 is typically `MONIT`. The number under this heading is used when conversion of the number to or from the fractional (%) uncertainty is required.
- L.6: Heading for the total uncertainty in the dependent variable (optional if L.7 is given). The heading specified in L.6 is typically `ERR-T` or `DATA-ERR`.
- L.7: Heading for the 1st partial uncertainty in the dependent variable (optional if L.6 is given). The heading specified in L.7 and below is typically `ERR-S`, `ERR-1`, `ERR-2` etc.
- L.8: Heading for the 2nd partial uncertainty in the dependent variable (optional)

etc.

The column 1 is reserved for a flag. It must be empty in Line 1 to 6 must be empty. In Line 7 and below, this column can be empty or

- "S": indicates the uncertainty must be ignored when the dataset is treated as a shape dataset (MAKCOV with option `-s`)
- "#": indicates it is a comment line (always ignored).

The column 2 is for the first character of a heading if a heading is given on the line.

When only a range of the partial uncertainty is given under ERR-ANALYS, its lower boundary is treated as a constant of the dataset. The user may add an extra uncertainty following an ad-hoc heading such as `ERR-A`, `ERR-B`.

Tabulation without estimation of correlation coefficients

When the only tabulation in the four-column (x, Δx, y, Δy) structure is necessary, L.5 always has a correlation flag "+". If the partial uncertainties are in EXFOR without the total uncertainty,

the partial uncertainties under the heading in L.6 and below must be flagged by "-". Δy is calculated by adding these partial uncertainties in % quadratically.

Example 1
Δx is calculated by the difference between EN-MIN and EN-MAX. Δy is taken from ERR-T.

```
1: EN-MIN
2: EN-MAX
3:
4: DATA
5:
6: ERR-T      +
```

Example 2
Δx is taken from EN-ERR. Δy is obtained by the quadrature sum of ERR-S, ERR-1 and ERR-2.

```
1: EN
2: EN
3: EN-ERR
4: DATA
5:
6:            +
7: ERR-S      0
8: ERR-1      0
9: ERR-2      0
```

Example 3
Δx is always zero. Δy is obtained by the quadrature sum of ERR-S, ERR-1 and MONIT-ERR. The MONIT-ERR coded in other than % (e.g., in barn) is divided by MONIT for conversion to the fractional (%) uncertainty. The flag "S" on the L.9 indicates that the MONIT-ERR value is ignored when the dataset is treated as a shape dataset (i.e., MAKCOV is used with the option -s).

```
1: EN
2: EN
3:
4: DATA
5: MONIT
6:            +
7: ERR-S      0
8: ERR-1      0
9:SMONIT-ERR  0
```

Tabulation with estimation of correlation coefficients

The correlation coefficients are calculated on the assumption that each partial uncertainty is fully correlated or uncorrelated between two data points (x_1,y_1) and (x_2,y_2).

- The correlation property of each partial uncertainty is specified by a flag 1 (fully correlated) or 0 (uncorrelated).

- When all partial uncertainties are given in EXFOR, then the heading of the total uncertainty should not be specified in L.6.

When the total uncertainty is known but not all partial uncertainties are known, the "residual uncertainty" (the difference of the quadrature sum of all known partial uncertainties) is calculated for which the user must assign a correlation property.

L6 is used to specify the heading of the total uncertainty, the "residual uncertainty" (the difference of the quadrature sum of all known partial uncertainties) is calculated. The heading of the total uncertainty specified in L.6 must be followed by blanks and one of the following flags:

*: Estimation of the correlation coefficients is skipped (e.g., when the correlation coefficients estimated by the experimentalists are available in EXFOR)

F: The residual uncertainty is fully correlated.

U: The residual uncertainty is uncorrelated.

L7 and below is used for the heading of a partial uncertainty, the heading must be followed by one of the following flags:

0: This partial uncertainty as uncorrelated. (currently not allowed when the total uncertainty heading is flagged with U)

1: This partial uncertainty is fully correlated. (currently not allowed when the total uncertainty heading is flagged with F)

-: This partial uncertainty is subtracted from the total uncertainty. (This is used only when the total uncertainty heading is specified in L.6. The 1st column must be flagged by S.)

Example 4
The partial uncertainty coded under `ERR-S` is specified as uncorrelated. The residual uncertainty is specified as fully correlated.

```
1: EN
2: EN
3: EN-ERR
4: DATA
5:
6: ERR-T       F
7: ERR-S       0
```

Example 5
The partial uncertainty coded under `ERR-1` and `ERR-2` is specified as fully uncorrelated. The residual uncertainty is specified as uncorrelated.

```
1: EN
2: EN
3: EN-ERR
4: DATA
5:
6: ERR-T       U
7: ERR-1       1
8: ERR-2       1
```

Example 6

The partial uncertainty coded under ERR-S is specified as uncorrelated, and those under ERR-1 and ERR-2 is specified as fully uncorrelated. The residual uncertainty is specified as uncorrelated.

```
1: EN
2: EN
3: EN-ERR
4: DATA
5:
6:
7: ERR-S      0
8: ERR-1      1
9: ERR-2      1
```

Example 7

Same as Example 6, but ERR-2 is ignored.

```
1: EN
2: EN
3: EN-ERR
4: DATA
5:
6:
7: ERR-S      0
8: ERR-1      1
9:#ERR-2      1
```

Example 8

Same as Example 6, but MONIT-ERR instead of ERR-2. The flag "S" on the L.9 indicates that the MONIT-ERR value is ignored when the dataset is treated as a shape dataset (i.e., MAKCOV is used with the option -s).

```
1: EN
2: EN
3: EN-ERR
4: DATA
5:
6:
7: ERR-S      0
8: ERR-1      1
9:SMONIT-ERR  1
```

Example 9

Same as Example 4, but L.8 indicates that the MONIT-ERR is subtracted from the total uncertainty when the dataset is treated as a shape dataset.

```
1: EN
2: EN
3: EN-ERR
4: DATA
5:
6: ERR-T      F
7: ERR-S      0
8:SMONIT-ERR  -
```

Example 10

The partial uncertainty coded under ERR-S is specified as uncorrelated, and a fully correlated partial uncertainty of 1.00% not in EXFOR is added with ERR-A in L.8.

```
1: EN
2: EN
3: EN-ERR
4: DATA
5:
6:
7: ERR-S       0
8: ERR-A       1   1.00 # Normalization uncertainty 1% added
```

Numerical example

Numerical examples are given for the correlation coefficient and total uncertainty Δy calculated with the various setting of correlation flags for a dataset consisting of two data points having the same total and partial uncertainties coded by

```
ERR-T      ERR-S       ERR-1       MONIT-ERR
PER-CENT   PER-CENT    PER-CENT    PER-CENT
 6.         4.          1.          2.
```

for which the L.6 to L.9 of the HED file are

```
6: ERR-T          ?
7: ERR-S          ?
8: ERR-1          ?
9:SMONIT-ERR      ?
```

The next table summarizes the obtained correlation coefficient between the two data points and the total uncertainty Δy for various combination of the flags. The first column shows use of -s option (treat as a shape dataset) in MAKCOV. "(#)" indicates that the line is commented out.

Shape	ERR-T	ERR-S	ERR-1	MONIT-ERR	Cor	Δy
No	U	1	1	1	0.583	6.000
Yes	U	1	1	1	0.531	5.657
No	F	0	0	(#)	0.528	6.000
No	U	(#)	1	1	0.139	6.000
Yes	U	(#)	1	1	0.031	5.657
No		0	1	1	0.238	4.583
Yes		0	1	1	0.059	4.123
No	F	0	0	-	0.528	6.000
Yes	F	0	0	-	0.469	5.657

Appendix 5: Use a ForEXy code as a module

The ForEXy package is available on the PyPI repository (https://pypi.org/project/forexy/), and can be installed by

```
pip install forexy
```

Then one can load the main function of each code as an external library. See "Arguments" of the each code description for the list of the arguments. The module must be called with all arguments in the same order.

An argument name starting from @ indicate that it must be given as a list. (e.g., `@key_keep` of POIPOI and X4TOJ4 and `@key_anal` of REFDOI).

Example 1

The following script (utilizing the ForEXy code REFBIB) reads an EXFOR reference code or DOI, and output a BibTeX entry:

```
from forexy import x4_refbib

def main():

  ref_inp=input("Input EXFOR reference code or DOI -> ")
  fauthor   = "any"
  file_dict = "dict.json"
  file_bib  = "exfor.bib"
  format    = "bibtex"
  email     = "email@address.com"
  force     = False
  strip     = False

  x4_refbib.main\
  (ref_inp,fauthor,file_dict,file_bib,format,email,force,strip)

  f=open(file_bib, 'r')
  lines=f.readlines()
  f.close

  for line in lines:
    print(line, end="")

  return

if __name__ == "__main__":
  main()
  exit()
```

Example 2

The following script (utilizing the ForEXy code X4TOJ4) reads an EXFOR file, and output a J4 file for the keywords AUTHOR, REFERENCE, FACILITY and REACTION:

```
from forexy import x4_x4toj4

def main():

  file_x4   =input("Input EXFOR file name -> ")
  file_dict = "dict.json"
  file_j4   = "exfor.json"
  key_keep  = ["AUTHOR", "REFERENCE", "FACILITY", "REACTION"]
  force     = "False"
  check     = "True"
  add19     = "True"
  flag      = "False"
  string    = "False"

  x4_x4toj4.main\
  (file_x4,file_dict,file_j4,key_keep,force,check,add19,flag,string)

  f=open(file_j4, 'r')
  lines=f.readlines()
  f.close

  for line in lines:
    print(line, end="")

  return

if __name__ == "__main__":
  main()
  exit()
```

Nuclear Data Section
International Atomic Energy Agency
P.O. Box 100
A-1400 Vienna
Austria

e-mail: nds.contact-point@iaea.org
fax: (43-1)26007
telephone: (43-1)2600-21710
web: https://nds.iaea.org/